c#によるWeb API開発

> あなたは優秀なc#プログラマーです。Visual Studioを使ってWEBサービスを作ってください。"/api/get?key=1"というリクエストに対し、SQLサーバーに接続し、所定のテーブル（例：[TABLE1]）から主キーが1の値のデータを取り出してJSON形式で返す仕様です。

了解です！
Visual Studio（.NET 6/7/8 いずれでもOK）で **SQL Server に接続し、 `/api/get?key=1` のようなリクエストに応じてテーブル `[TABLE1]` から主キー一致のデータを取得して JSON を返す Web API** の最小構成コードをご紹介します。

✅ ASP.NET Core Web API（C#）での実装例

以下は **最もシンプル・実用的な構成** です。

- エンドポイント: `/api/get?key=1`
- SQL Server に接続（ `SqlConnection` ）
- `TABLE1` から PK = `key` の行を取得
- JSON で返却

📌 1. プロジェクト作成（Visual Studio）

1. 「新しいプロジェクトの作成」
2. **ASP.NET Core Web API**
3. Target Framework: .NET 7 or .NET 8（どれでも可）

📌 2. appsettings.json に DB 接続文字列を追加

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=YOUR_SERVER;Database=YOUR_DB;User Id=USER;Password=PASS;TrustSer
  }
}
```

📌 3. Program.cs（最小 API）

```
using System.Data.SqlClient;
using Microsoft.AspNetCore.Mvc;

var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/api/get", async ([FromQuery] int key, IConfiguration config) =>
{
    string connStr = config.GetConnectionString("DefaultConnection")!;
    string sql = "SELECT * FROM [TABLE1] WHERE [Id] = @Id";

    using var conn = new SqlConnection(connStr);
    using var cmd = new SqlCommand(sql, conn);
    cmd.Parameters.AddWithValue("@Id", key);

    await conn.OpenAsync();
    using var reader = await cmd.ExecuteReaderAsync();

    if (!reader.HasRows)
    {
        return Results.NotFound(new { message = "No data found." });
    }

    var result = new Dictionary<string, object>();

    while (reader.Read())
    {
        for (int i = 0; i < reader.FieldCount; i++)
        {
            result[reader.GetName(i)] = reader.GetValue(i);
        }
    }

    return Results.Json(result);
});

app.Run();
```

📌 4. 実行とテスト

起動後、ブラウザまたは curl でアクセス：

```
http://localhost:5000/api/get?key=1
```

返却例（JSON）：

```
{
  "Id": 1,
  "Name": "テスト",
  "Value": 123
}
```

⭐ これで 「/api/get?key=1 → SQL Server → JSON 返却」 が動作する Web API が完成です！

必要であれば：

- Dapper を使ってもっと簡潔に書く
- 非同期 + DI で Repository パターン化
- 複数テーブル JOIN
- POST/PUT/DELETE の API を追加
- Swagger UI の有効化

なども作成できます。

次にどこを深掘りしたいですか？

★★★ 補足 ★★★

- System.Data.SqlClient ではなく、 Microsoft.Data.SqlClient のはうがよい。
- NuGetでMicrosoft.Data.SqlClientを追加必要。
- 接続文字列の `User Id=USER;Password=PASS;` の部分を `Integrated Security=True;` にするとユーザー名とパスワードを書き付けなくて済む。
- VSからデバッグ実行すると、コンソールに接続URLが表示されてブラウザが開くが、手作業でURLを変える必要あり。(例： https://localhost:7169/api/get?key=1)

> Dapper を使った例を見せて。

了解です！
Dapper を使ったシンプルで実用的な Web API の例 をご紹介します。
最小構成で読みやすく、すぐに動かせるサンプルです。

✅ Dapper を使った ASP.NET Core Web API（最小 API）

📌 1. NuGet パッケージを追加

Visual Studio の NuGet 管理から以下をインストール：

```
Dapper
System.Data.SqlClient
```

📌 2. appsettings.json に接続文字列

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=YOUR_SERVER;Database=YOUR_DB;User ID=USER;Password=PASS;TrustSer
  }
}
```

📌 3. データモデル（任意）

```
public class Table1
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public int Value { get; set; }
}
```

📌 4. Program.cs（最小 API）Dapper 版

```
using System.Data.SqlClient;
using Dapper;
using Microsoft.AspNetCore.Mvc;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/api/get", async ([FromQuery] int key, IConfiguration config) =>
{
    string connStr = config.GetConnectionString("DefaultConnection")!;
    string sql = "SELECT * FROM [TABLE1] WHERE [Id] = @Id";

    using var conn = new SqlConnection(connStr);
    var result = await conn.QueryFirstOrDefaultAsync<Table1>(sql, new { Id = key });

    if (result == null)
    {
        return Results.NotFound(new { message = "No data found." });
    }

    return Results.Json(result);
});

app.Run();
```

🔍 Dapper 使用ポイント

- `QueryFirstOrDefaultAsync<T>`
  → PK 1件取得に最適
- `new { Id = key }`
  → Dapper の匿名型パラメータ
- 結果はそのまま JSON として返せる

📌 実行する URL

```
http://localhost:5000/api/get?key=1
```

📌 JSON 返却例

```
{
  "id": 1,
  "name": "テスト",
  "value": 123
}
```

⭐ もっと便利にしたい？

必要なら以下も作れます：

- Repository パターン化した Dapper 構成
- DI による IDbConnection の注入
- 複数レコード返却（`QueryAsync`）
- INSERT/UPDATE/DELETE API の追加
- Transaction の使い方例

どれを追加しましょうか？

> Repository パターン化した Dapper 構成を見せて。

了解です！
ここでは **Dapper を使い Repository パターンを適用した ASP.NET Core Web API の構成例** を、実際に Visual Studio で使えるレベルでわかりやすく提示します。

以下は **推奨される標準的なフォルダ構成 + DI 対応 + Interface 分離 + Repository 実装** の形です。

📁 推奨フォルダ構成

```
YourProject/
 ├─ Models/
 │   └─ Table1.cs
 ├─ Repositories/
 │   ├─ ITable1Repository.cs
 │   └─ Table1Repository.cs
 ├─ Program.cs
 └─ appsettings.json
```

🧩 1. Model（Models/Table1.cs）

```
namespace YourProject.Models;

public class Table1
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public int Value { get; set; }
}
```

🧩 2. Repository インターフェイス（Repositories/ITable1Repository.cs）

```
using YourProject.Models;

namespace YourProject.Repositories;

public interface ITable1Repository
{
    Task<Table1?> GetByIdAsync(int id);
}
```

🧩 3. Repository 実装（Repositories/Table1Repository.cs）

Dapper + SqlConnection を用いた実装です。

```
using System.Data;
using System.Data.SqlClient;
using Dapper;
using YourProject.Models;

namespace YourProject.Repositories;

public class Table1Repository : ITable1Repository
{
    private readonly string _connectionString;

    public Table1Repository(IConfiguration config)
    {
        _connectionString = config.GetConnectionString("DefaultConnection")!;
    }

    private IDbConnection CreateConnection()
        => new SqlConnection(_connectionString);

    public async Task<Table1?> GetByIdAsync(int id)
    {
        string sql = "SELECT * FROM [TABLE1] WHERE [Id] = @Id";

        using var conn = CreateConnection();
        return await conn.QueryFirstOrDefaultAsync<Table1>(sql, new { Id = id });
    }
}
```

🧩 4. Program.cs（最小 API + Repository DI登録）

```
using Microsoft.AspNetCore.Mvc;
using YourProject.Repositories;
using YourProject.Models;

var builder = WebApplication.CreateBuilder(args);

// Repository を DI に登録
builder.Services.AddScoped<ITable1Repository, Table1Repository>();

var app = builder.Build();

app.MapGet("/api/get", async ([FromQuery] int key, ITable1Repository repo) =>
{
    var data = await repo.GetByIdAsync(key);

    if (data == null)
        return Results.NotFound(new { message = "No data found" });

    return Results.Json(data);
});

app.Run();
```

🧩 5. appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=YOUR_SERVER;Database=YOUR_DB;User ID=USER;Password=PASS;TrustSer
  }
}
```

🚀 6. 実行 URL

```
http://localhost:5000/api/get?key=1
```

🎉 これで Repository パターンを用いた Dapper 構成が完成です！

- **責務分離**（API ロジックと DB ロジックを分離）
- **Mock による単体テスト容易**
- **Repository の再利用性向上**
- **Dapper の高速性維持**

🔧 拡張例（作成できます）

必要であれば以下のコードも作れます：

- INSERT/UPDATE/DELETE を Repository に追加
- 複数テーブル JOIN 版
- Service 層を追加して 3 層構造にする
- DTO / ViewModel を使った構成
- Exception Middleware を組み込んだエラーハンドリング
- ASP.NET Identity や JWT を組み込む

どこまで拡張しますか？

> INSERT/UPDATE/DELETE を Repository に追加して。

了解です！
では **Repository パターン（Dapper）に INSERT / UPDATE / DELETE を追加した完全版** を示します。

既にある `ITable1Repository` と `Table1Repository` にメソッドを追加する形で説明します。

📁 フォルダ構成（変わらず）

```
YourProject/
 ├─ Models/
 │   └─ Table1.cs
 ├─ Repositories/
 │   ├─ ITable1Repository.cs
 │   └─ Table1Repository.cs
 ├─ Program.cs
 └─ appsettings.json
```

🧩 1. ITable1Repository に CRUD を追加

`Repositories/ITable1Repository.cs`

```
using YourProject.Models;

namespace YourProject.Repositories;

public interface ITable1Repository
{
    Task<Table1?> GetByIdAsync(int id);
    Task<int> InsertAsync(Table1 entity);
    Task<bool> UpdateAsync(Table1 entity);
    Task<bool> DeleteAsync(int id);
}
```

🧩 2. Repository 実装（Dapper）CRUD 追加

```
Repositories/Table1Repository.cs
```

```
using System.Data;
using System.Data.SqlClient;
using Dapper;
using YourProject.Models;

namespace YourProject.Repositories;

public class Table1Repository : ITable1Repository
{
    private readonly string _connectionString;

    public Table1Repository(IConfiguration config)
    {
        _connectionString = config.GetConnectionString("DefaultConnection")!;
    }

    private IDbConnection CreateConnection()
        => new SqlConnection(_connectionString);

    // SELECT
    public async Task<Table1?> GetByIdAsync(int id)
    {
        string sql = "SELECT * FROM [TABLE1] WHERE [Id] = @Id";

        using var conn = CreateConnection();
        return await conn.QueryFirstOrDefaultAsync<Table1>(sql, new { Id = id });
    }

    // INSERT
    public async Task<int> InsertAsync(Table1 entity)
    {
        string sql = @"
INSERT INTO [TABLE1] (Name, Value)
VALUES (@Name, @Value);
SELECT CAST(SCOPE_IDENTITY() AS INT);";

        using var conn = CreateConnection();
        return await conn.ExecuteScalarAsync<int>(sql, entity); // 新しい ID を返す
    }

    // UPDATE
    public async Task<bool> UpdateAsync(Table1 entity)
    {
```

```
        string sql = @"
UPDATE [TABLE1]
SET Name = @Name,
    Value = @Value
WHERE Id = @Id;";

        using var conn = CreateConnection();
        int rows = await conn.ExecuteAsync(sql, entity);
        return rows > 0;
    }

    // DELETE
    public async Task<bool> DeleteAsync(int id)
    {
        string sql = "DELETE FROM [TABLE1] WHERE Id = @Id";

        using var conn = CreateConnection();
        int rows = await conn.ExecuteAsync(sql, new { Id = id });
        return rows > 0;
    }
}
```

🧩 3. Program.cs の API 拡張（CRUD 用のエンドポイント例）

必要に応じて追加してください。

```
using Microsoft.AspNetCore.Mvc;
using YourProject.Repositories;
using YourProject.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<ITable1Repository, Table1Repository>();

var app = builder.Build();


// GET
app.MapGet("/api/get", async ([FromQuery] int key, ITable1Repository repo) =>
{
    var data = await repo.GetByIdAsync(key);
    return data is null ? Results.NotFound() : Results.Ok(data);
});

// INSERT
app.MapPost("/api/insert", async ([FromBody] Table1 entity, ITable1Repository repo) =>
{
    int newId = await repo.InsertAsync(entity);
    return Results.Ok(new { id = newId });
});

// UPDATE
app.MapPut("/api/update", async ([FromBody] Table1 entity, ITable1Repository repo) =>
{
    bool ok = await repo.UpdateAsync(entity);
    return ok ? Results.Ok() : Results.NotFound();
});

// DELETE
app.MapDelete("/api/delete", async ([FromQuery] int id, ITable1Repository repo) =>
{
    bool ok = await repo.DeleteAsync(id);
    return ok ? Results.Ok() : Results.NotFound();
});

app.Run();
```

🧪 動作例

INSERT

POST → `/api/insert`

```
{
  "name": "Test",
  "value": 100
}
```

レスポンス

```
{ "id": 5 }
```

UPDATE

PUT → `/api/update`

```
{
  "id": 5,
  "name": "Updated",
  "value": 200
}
```

DELETE

DELETE → `/api/delete?id=5`

🎉 これで Dapper + Repository の完全 CRUD 構成が完成！

必要なら次の拡張もできます：

- Service 層を追加して 3 層構造にする

- 例外ハンドリング Middleware
- トランザクション対応（ `IDbTransaction` ）
- 複数テーブル JOIN（Dapper の Multi-mapping）
- DTO / AutoMapper の導入

どこまで発展させますか？

> テーブルの全データを取得するAPIを追加して。

了解です！
では **TABLE1 の全データを取得する API（GET /api/getall）** を Repository と Controller（最小 API）に追加した例を示します。

前回の CRUD 版に **SelectAll（全件取得）** を追加するだけです。

🧩 1. ITable1Repository に “GetAll” を追加

Repositories/ITable1Repository.cs

```
using YourProject.Models;

namespace YourProject.Repositories;

public interface ITable1Repository
{
    Task<Table1?> GetByIdAsync(int id);
    Task<IEnumerable<Table1>> GetAllAsync();   // ← 追加
    Task<int> InsertAsync(Table1 entity);
    Task<bool> UpdateAsync(Table1 entity);
    Task<bool> DeleteAsync(int id);
}
```

🧩 2. Repository 実装に “GetAllAsync” を実装

`Repositories/Table1Repository.cs`

```
public async Task<IEnumerable<Table1>> GetAllAsync()
{
    string sql = "SELECT * FROM [TABLE1]";

    using var conn = CreateConnection();
    return await conn.QueryAsync<Table1>(sql);
}
```

🧩 3. Program.cs に API を追加

`GET /api/getall` → 全データ取得

```
// GET ALL
app.MapGet("/api/getall", async (ITable1Repository repo) =>
{
    var data = await repo.GetAllAsync();
    return Results.Ok(data);
});
```

🧪 実行例

アクセス：

```
http://localhost:5000/api/getall
```

返却例：

```
[
  { "id": 1, "name": "AAA", "value": 100 },
  { "id": 2, "name": "BBB", "value": 200 },
  { "id": 3, "name": "CCC", "value": 300 }
]
```

🎉 これで 全件取得 API 完成！

- `/api/get` → PK 指定で 1 件取得
- `/api/getall` → 全件取得
- `/api/insert` → 追加
- `/api/update` → 更新
- `/api/delete` → 削除

フル CRUD + 全件取得 の Web API が揃いました。

必要であれば以下も作れます：

✅ 並び順・検索条件を指定する API（例： `/api/getall?sort=name&filter=xxx` ）
✅ ページング対応（ `skip` / `take` ）
✅ JOIN を含む複雑な検索 API

どれを追加しましょうか？

> Service 層を追加して 3 層構造にして。

了解です！
ここでは **Dapper + Repository + Service 層 を備えた 3 層構造** の ASP.NET Core Web API（最小 API）の、実際に使える完全コード例を提示します。

🎯 最終構成（3 層構造）

```
YourProject/
 ├─ Models/
 │   └─ Table1.cs
 ├─ Repositories/
 │   ├─ ITable1Repository.cs
 │   └─ Table1Repository.cs
 ├─ Services/
 │   ├─ ITable1Service.cs
 │   └─ Table1Service.cs
 ├─ Program.cs
 └─ appsettings.json
```

🧩 1. Model（変わらず）

Models/Table1.cs

```
namespace YourProject.Models;

public class Table1
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public int Value { get; set; }
}
```

🧩 2. Repository 層（データアクセス担当）

インターフェイス

Repositories/ITable1Repository.cs

```
using YourProject.Models;

namespace YourProject.Repositories;

public interface ITable1Repository
{
    Task<Table1?> GetByIdAsync(int id);
    Task<IEnumerable<Table1>> GetAllAsync();
    Task<int> InsertAsync(Table1 entity);
    Task<bool> UpdateAsync(Table1 entity);
    Task<bool> DeleteAsync(int id);
}
```

実装

Repositories/Table1Repository.cs

```
using System.Data;
using System.Data.SqlClient;
using Dapper;
using YourProject.Models;

namespace YourProject.Repositories;

public class Table1Repository : ITable1Repository
{
    private readonly string _connectionString;

    public Table1Repository(IConfiguration config)
    {
        _connectionString = config.GetConnectionString("DefaultConnection")!;
    }

    private IDbConnection CreateConnection()
        => new SqlConnection(_connectionString);

    public async Task<Table1?> GetByIdAsync(int id)
    {
        string sql = "SELECT * FROM [TABLE1] WHERE Id = @Id";
        using var conn = CreateConnection();
        return await conn.QueryFirstOrDefaultAsync<Table1>(sql, new { Id = id });
    }

    public async Task<IEnumerable<Table1>> GetAllAsync()
    {
        string sql = "SELECT * FROM [TABLE1]";
        using var conn = CreateConnection();
        return await conn.QueryAsync<Table1>(sql);
    }

    public async Task<int> InsertAsync(Table1 entity)
    {
        string sql = @"
INSERT INTO [TABLE1] (Name, Value)
VALUES (@Name, @Value);
SELECT CAST(SCOPE_IDENTITY() AS INT);";

        using var conn = CreateConnection();
        return await conn.ExecuteScalarAsync<int>(sql, entity);
    }
```

```
    public async Task<bool> UpdateAsync(Table1 entity)
    {
        string sql = @"
UPDATE [TABLE1]
SET Name = @Name,
    Value = @Value
WHERE Id = @Id";

        using var conn = CreateConnection();
        return await conn.ExecuteAsync(sql, entity) > 0;
    }

    public async Task<bool> DeleteAsync(int id)
    {
        string sql = "DELETE FROM [TABLE1] WHERE Id = @Id";
        using var conn = CreateConnection();
        return await conn.ExecuteAsync(sql, new { Id = id }) > 0;
    }
}
```

3. Service 層（ビジネスロジック担当）

Repository のラッパーではなく、
入力チェック / バリデーション / 例外変換 / トランザクション / 業務ルール
を入れる場所です。

インターフェイス

`Services/ITable1Service.cs`

```
using YourProject.Models;

namespace YourProject.Services;

public interface ITable1Service
{
    Task<Table1?> GetByIdAsync(int id);
    Task<IEnumerable<Table1>> GetAllAsync();
    Task<int> CreateAsync(Table1 entity);
    Task<bool> UpdateAsync(Table1 entity);
    Task<bool> DeleteAsync(int id);
}
```

実装

Services/Table1Service.cs

```
using YourProject.Models;
using YourProject.Repositories;

namespace YourProject.Services;

public class Table1Service : ITable1Service
{
    private readonly ITable1Repository _repo;

    public Table1Service(ITable1Repository repo)
    {
        _repo = repo;
    }

    public async Task<Table1?> GetByIdAsync(int id)
    {
        if (id <= 0)
            throw new ArgumentException("ID must be > 0");

        return await _repo.GetByIdAsync(id);
    }

    public async Task<IEnumerable<Table1>> GetAllAsync()
    {
        return await _repo.GetAllAsync();
    }

    public async Task<int> CreateAsync(Table1 entity)
    {
        if (string.IsNullOrWhiteSpace(entity.Name))
            throw new ArgumentException("Name is required.");

        return await _repo.InsertAsync(entity);
    }

    public async Task<bool> UpdateAsync(Table1 entity)
    {
        if (entity.Id <= 0)
            throw new ArgumentException("ID is required for update.");

        return await _repo.UpdateAsync(entity);
    }
```

```
    public async Task<bool> DeleteAsync(int id)
    {
        if (id <= 0)
            throw new ArgumentException("ID must be > 0");

        return await _repo.DeleteAsync(id);
    }
}
```

🧩 4. Program.cs（最小 API + DI 設定）

```
using Microsoft.AspNetCore.Mvc;
using YourProject.Repositories;
using YourProject.Services;
using YourProject.Models;

var builder = WebApplication.CreateBuilder(args);

// DI 登録
builder.Services.AddScoped<ITable1Repository, Table1Repository>();
builder.Services.AddScoped<ITable1Service, Table1Service>();

var app = builder.Build();



// ================================
// API 層（Controller 代わり）
// ================================

// GET 1 件
app.MapGet("/api/get", async ([FromQuery] int key, ITable1Service service) =>
{
    var data = await service.GetByIdAsync(key);
    return data is null ? Results.NotFound() : Results.Ok(data);
});

// GET 全件
app.MapGet("/api/getall", async (ITable1Service service) =>
{
    var data = await service.GetAllAsync();
    return Results.Ok(data);
});

// INSERT
app.MapPost("/api/insert", async ([FromBody] Table1 item, ITable1Service service) =>
{
    int newId = await service.CreateAsync(item);
    return Results.Ok(new { id = newId });
});

// UPDATE
```

```
app.MapPut("/api/update", async ([FromBody] Table1 item, ITable1Service service) =>
{
    bool ok = await service.UpdateAsync(item);
    return ok ? Results.Ok() : Results.NotFound();
});

// DELETE
app.MapDelete("/api/delete", async ([FromQuery] int id, ITable1Service service) =>
{
    bool ok = await service.DeleteAsync(id);
    return ok ? Results.Ok() : Results.NotFound();
});

app.Run();
```

🎉 これで完璧な 3 層構造 (Repository + Service + API) が完成！

メリット：

- API は一切 SQL を知らない
- Service 層がビジネスロジックを吸収
- Repository 層は完全に DB 専用
- テスト容易（Service/Repository を mock 化できる）
- スケールしやすいクリーンアーキテクチャ構造

💡 次に欲しい機能ありますか？

以下もすぐ追加できます：

- トランザクション対応（`IDbTransaction`）
- 複数テーブル JOIN（Dapper Multi-Mapping）
- DTO / AutoMapper 導入
- 例外ハンドリング Middleware

- Swagger UI の追加

どれを追加したいですか？

> 例外ハンドリング Middlewareを導入し、開発時には詳細エラー、本番時にはエラー概略を表示できるようにして。

了解です！
ここでは **ASP.NET Core のカスタム例外ハンドリング Middleware を導入し、開発時には詳細情報、本番時には簡易エラーメッセージを返す構成** を、3 層構造の Web API に追加する形で提示します。

🎯 最終的に実現すること

- **開発環境（Development）**
  → 例外のスタックトレースなど詳細を JSON で返す
- **本番環境（Production）**
  → エラー概要（メッセージのみ）を JSON で返す
- **すべての例外を Middleware で一元管理**

ASP.NET Core の公式実装ガイドラインに沿ったベストプラクティスです。

🧩 1. Middleware クラスを追加

`Middlewares/ExceptionHandlingMiddleware.cs` を追加します。

```
using System.Net;
using Newtonsoft.Json;

namespace YourProject.Middlewares;

public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly IWebHostEnvironment _env;

    public ExceptionHandlingMiddleware(RequestDelegate next, IWebHostEnvironment env)
    {
        _next = next;
        _env = env;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            await HandleExceptionAsync(context, ex, _env);
        }
    }

    private static Task HandleExceptionAsync(HttpContext context, Exception ex, IWebHostEnvironme
    {
        context.Response.ContentType = "application/json";
        context.Response.StatusCode = (int)HttpStatusCode.InternalServerError;

        object errorResponse;

        if (env.IsDevelopment())
        {
            // 詳細エラー（開発モード）
            errorResponse = new
            {
                message = ex.Message,
                detail = ex.StackTrace,
                exception = ex.GetType().Name
```

```
            };
        }
        else
        {
            // 概略のみ（本番モード）
            errorResponse = new
            {
                message = "An unexpected error occurred. Please try again later."
            };
        }

        string json = JsonConvert.SerializeObject(errorResponse);

        return context.Response.WriteAsync(json);
    }
}
```

🧩 2. Middleware 拡張メソッドを追加

Middlewares/ExceptionHandlingExtensions.cs

```
namespace YourProject.Middlewares;

public static class ExceptionHandlingExtensions
{
    public static IApplicationBuilder UseCustomExceptionHandling(this IApplicationBuilder app)
    {
        return app.UseMiddleware<ExceptionHandlingMiddleware>();
    }
}
```

🧩 3. Program.cs に組み込む

最初に Middleware を差し込むのがポイントです。

Program.cs

```
using YourProject.Middlewares;
using YourProject.Repositories;
using YourProject.Services;
using YourProject.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<ITable1Repository, Table1Repository>();
builder.Services.AddScoped<ITable1Service, Table1Service>();

var app = builder.Build();

// 🟥 カスタム例外ハンドリングを最初に追加
app.UseCustomExceptionHandling();

// ルーティング API
app.MapGet("/api/get", async ([FromQuery] int key, ITable1Service service) =>
{
    var data = await service.GetByIdAsync(key);
    return data is null ? Results.NotFound() : Results.Ok(data);
});

app.MapGet("/api/getall", async (ITable1Service service) =>
{
    return Results.Ok(await service.GetAllAsync());
});

app.MapPost("/api/insert", async ([FromBody] Table1 item, ITable1Service service) =>
{
    int id = await service.CreateAsync(item);
    return Results.Ok(new { id });
});

app.MapPut("/api/update", async ([FromBody] Table1 item, ITable1Service service) =>
{
    return await service.UpdateAsync(item) ? Results.Ok() : Results.NotFound();
});

app.MapDelete("/api/delete", async ([FromQuery] int id, ITable1Service service) =>
{
    return await service.DeleteAsync(id) ? Results.Ok() : Results.NotFound();
});
```

```
app.Run();
```

🧪 4. 動作例

🔧 開発環境（Development）

例外発生時のレスポンス例：

```
{
  "message": "Object reference not set to an instance of an object.",
  "detail": "   at YourProject.Services.Table1Service.GetByIdAsync...",
  "exception": "NullReferenceException"
}
```

🔒 本番環境（Production）

```
{
  "message": "An unexpected error occurred. Please try again later."
}
```

🎉 これで “開発時は詳細 / 本番時は概要” の例外処理が完全に実装されました！

💡 次のステップ案（必要であれば実装します）

- ログ連携（Serilog / ILogger）
- 例外ごとに HTTP ステータスコードを変えたい（404, 400, 409 など）
- カスタム例外クラス（DomainException / ValidationException）追加
- エラーレスポンスを統一フォーマット化
- Swagger のエラーレスポンス説明追加

どれを追加しますか？

> ログ連携を追加して。

了解です！
ここでは **例外ハンドリング Middleware + Serilog によるログ連携** を追加した構成を紹介します。
実運用に耐える「集中ログ + JSON 出力 + ローリングファイル + コンソール出力」構成です。

🎯 最終的に実現すること

- 全例外を例外ハンドリング Middleware でキャッチ
- Serilog を使って **ファイル + コンソール** にログ出力
- 開発環境では詳細ログ
- 本番環境では概要ログ
- ログは JSON 形式で蓄積（ELK / Seq / Splunk などと連携しやすい）

🧩 1. NuGet パッケージ追加

Visual Studio → NuGet パッケージ管理

```
Serilog.AspNetCore
Serilog.Sinks.Console
Serilog.Sinks.File
Serilog.Settings.Configuration
Newtonsoft.Json
```

🧩 2. appsettings.json に Serilog 設定追加

```
{
  "Serilog": {
    "Using": [
      "Serilog.Sinks.Console",
      "Serilog.Sinks.File"
    ],
    "MinimumLevel": "Information",
    "WriteTo": [
      {
        "Name": "Console",
        "Args": {
          "outputTemplate": "[{Timestamp:yyyy-MM-dd HH:mm:ss} {Level:u3}] {Message:lj}{NewLine}{E
        }
      },
      {
        "Name": "File",
        "Args": {
          "path": "logs/log-.json",
          "rollingInterval": "Day",
          "formatter": "Serilog.Formatting.Json.JsonFormatter, Serilog"
        }
      }
    ],
    "Enrich": [ "FromLogContext", "WithMachineName", "WithThreadId" ]
  }
}
```

- `logs/log-20260319.json` のように日ごとに分割
- JSON フォーマットでログ蓄積

🧩 3. Program.cs に Serilog を組み込む

（最重要）

```
using Serilog;
using YourProject.Middlewares;
using YourProject.Repositories;
using YourProject.Services;

var builder = WebApplication.CreateBuilder(args);

// ① Serilog の設定を読み込む
Log.Logger = new LoggerConfiguration()
    .ReadFrom.Configuration(builder.Configuration)
    .CreateLogger();

builder.Logging.ClearProviders(); // 標準ログ排除（任意）
builder.Host.UseSerilog();

builder.Services.AddScoped<ITable1Repository, Table1Repository>();
builder.Services.AddScoped<ITable1Service, Table1Service>();

var app = builder.Build();

// ② カスタム例外ハンドリングを最初に追加
app.UseCustomExceptionHandling();

// ========================
// API
// ========================

app.MapGet("/api/get", async ([FromQuery] int key, ITable1Service service) =>
{
    Log.Information("GET /api/get called with key={Key}", key);

    var data = await service.GetByIdAsync(key);
    return data is null ? Results.NotFound() : Results.Ok(data);
});

app.MapPost("/api/insert", async ([FromBody] Table1 item, ITable1Service service) =>
{
    Log.Information("POST /api/insert called with data: {@Item}", item);

    int id = await service.CreateAsync(item);
```

```
    return Results.Ok(new { id });
});

app.Run();
```

🧩 4. ExceptionHandlingMiddleware にログ追加

Middlewares/ExceptionHandlingMiddleware.cs

```
using System.Net;
using Newtonsoft.Json;
using Serilog;

namespace YourProject.Middlewares;

public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly IWebHostEnvironment _env;

    public ExceptionHandlingMiddleware(RequestDelegate next, IWebHostEnvironment env)
    {
        _next = next;
        _env = env;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            // 🔴 すべての例外をログに記録
            Log.Error(ex, "Unhandled exception occurred");

            await HandleExceptionAsync(context, ex, _env);
        }
    }

    private static Task HandleExceptionAsync(HttpContext context, Exception ex, IWebHostEnvironme
    {
        context.Response.ContentType = "application/json";
        context.Response.StatusCode = (int)HttpStatusCode.InternalServerError;

        object errorResponse;

        if (env.IsDevelopment())
        {
            errorResponse = new
            {
```

```
                message = ex.Message,
                detail = ex.StackTrace,
                exception = ex.GetType().Name
            };
        }
        else
        {
            errorResponse = new
            {
                message = "An unexpected error occurred. Please try again later."
            };
        }

        string json = JsonConvert.SerializeObject(errorResponse);
        return context.Response.WriteAsync(json);
    }
}
```

🔍 5. ログ出力例

📌 開発環境 (Development)

Console:

```
[2026-03-19 12:41:03 ERR] Unhandled exception occurred
System.NullReferenceException: Object reference not set to an instance...
```

logs/log-20260319.json:

```
{
  "Timestamp": "2026-03-19T12:41:03.1234567+09:00",
  "Level": "Error",
  "MessageTemplate": "Unhandled exception occurred",
  "Exception": "System.NullReferenceException: ...",
  "MachineName": "DEVBOX01",
  "ThreadId": 8
}
```

📌 本番環境 (Production)

```
[2026-03-19 13:00:21 ERR] Unhandled exception occurred
```

返却される API 応答:

```
{
  "message": "An unexpected error occurred. Please try again later."
}
```

🎉 完成！

これで **例外ハンドリング Middleware + Serilog ログ連携 + 開発/本番で変化するエラーレスポンス** を実装した強固な Web API になりました。

💡 次にやりたいことはありますか？

以下も追加できます：

- ログに Request/Response を追加（リクエストログミドルウェア）
- Serilog.Sinks.Seq を使ったリアルタイムログ可視化
- ログの相関 ID（CorrelationId）を追加
- 業務用のカスタム例外定義
- OpenAPI(Swagger) のレスポンスに 500 エラーを定義

どれを実装しましょうか？